

Unit

2

Lesson

2



Innovators in Action

The Brain Behind the Sound!

Time Frame: [90 min]

Lesson Objectives:

By the end of this lesson students will be able to:

- Recognize how robots help people at home, in schools, and in hospitals.
- Understand that the Arduino is like the brain that tells the talking device what to do.
- Use block coding to program the Arduino.
- Make a talking device that plays different sounds when you press different buttons.

Challenge questions of the lesson

1. How do robots help people in real life — at home, in school, or in hospitals?
2. What is the brain of a robot, and how does it tell the robot what to do?
3. Can we give our talking machine different voices or messages using code?
4. How can we control our machine using buttons, just like we control things at home?

SEL Relation

- Responsible Decision-Making: Choosing how to use technology to help people and solve real-world problems.
- Self-Management: Staying focused while learning and applying new coding skills step-by- step.
- Relationship Skills: Collaborating in teams to plan, code, and build a talking device.
- Social Awareness: Understanding how assistive devices support people with communication needs.
- Self-Awareness: Recognizing your own ability to solve problems with creativity and logic.

Engage



Teacher's Role:

- Read the dialogue with expression to spark curiosity.
- Prompt discussion using the mystery of the talking box.
- Ask students: “What’s inside?” “How could it talk?”
- Use images and clues (button, speaker, wires) to guide thinking.
- Facilitate reflection through the “Let’s Think” question, linking input to output logic.

Expected from Students:

- Listen carefully and follow the storyline.
- Share predictions about what’s inside the box.
- Discuss their ideas about how a button could trigger sound.
- Answer and draw/write their ideas in the Let’s Think section.

Parts of the books included.



Story



1. Create a Sense of Discovery.

- Stand at the front with a small box, button, or speaker prop in hand. Say with excitement:
 - “Class, do you remember the talking box from last time? Today, we’re going to open it up and find out what’s inside! What do you think helps it talk?”

2. Read the Story Aloud with Expression.

- Use character voices:
 - Adam = curious and surprised
 - Lila = thoughtful and guessing
 - Ms. Sara = calm, confident guide
- Pause to point to the box or gesture opening it up when Ms. Sara says she opens the box.
- Let students join in with a sound effect or say “Whoa!” with Adam to build energy.

3. Think Aloud and Prompt Discussion

- Pause and ask:
 - “Hmm... Ms. Sara called this little blue board the brain. Why do you think she said that?”
 - “Have you ever used something that talks when you press a button? What do you think is inside?”

4. Make the Learning Personal

- Show a drawing or real-life example of a circuit or microcontroller (like an Arduino or talking toy). Ask:
 - “Do you think we can give it new words today?”
 - Let students guess what they’d want the box to say!

5. Lead into the Think Activity

- Hand out the Think page. Say:
 - “Before we build, let’s stop and think: Why does the box only talk when we press the button? Use the pictures for clues, then write or draw your idea!”

Possible Strategies for Implementation:

1. Use Props or Real Components

- Bring a small “mystery box” with a button, speaker, or wires (even if non-working) to pass around.
- Let students hold or observe the parts closely to build curiosity.

2. Story Acting & Role Play

- Assign roles to students: Adam, Lila, Omar, Ms. Sara.
- Have them read or act out the dialogue with simple props or hand gestures.
- Use expressions like “press the button” or “open the box” with real motion.

3. Visual Supports

- Display the story dialogue on the board or use printed cards with bold character names and emojis.
- Show visual aids: speaker icons, Arduino, wires, “speech bubble” showing the word “Hello.”

4. Think-Pair-Share

- After the teacher asks: “Why does the box only talk when we press the button?”—
 - Give students 30 seconds to think quietly.
 - Pair them to share ideas with a partner.
 - Ask 2–3 pairs to share aloud with the class.

5. Concept Anchor Chart

- Create a class chart titled “What Makes the Box Talk?”
- List ideas students suggest under categories: Power, Sound, Brain/Controller.
- Revisit this chart in the Explain and Elaborate phases.

Let's Think!



Objective:

Students will investigate cause-and-effect relationships in a simple interactive circuit, focusing on how a button activates a response in a sound system. This builds foundational understanding of inputs (button) and outputs (speaker) in programmable hardware.

Possible answers:

- The box talks because the button sends a signal when pressed.
- Pressing the button starts the program or wakes the Arduino.
- The speaker only gets power when the button is pressed.
- The message is programmed to play when the button is activated.
- The code tells it to say a word only after the button is pushed.

These answers reflect a growing understanding of logic-based actions: input (button press) → processor (Arduino/code) → output (speaker).

Possible Strategies for implementation

1. Think-Pair-Share

- Students first think quietly, then discuss with a partner, and finally share their ideas aloud.

2. Sketch It Out

- Invite students to draw a quick sketch of what they imagine is happening inside the box.

3. Anchor Chart

- Start a chart titled “What Makes It Talk?” — add student ideas to revisit later during coding.

4. Vocabulary Boost

- Introduce or highlight words like input, signal, brain (Arduino), and code in context.

Explorer's Toolkit



Teacher's Role:

- Guide students to open Tinkercad and use a circuit with a button and LED.
- Explain the digital environment and basic controls.
- Help them make and observe cause–effect relationships.
- Ask probing questions like: “What happened when you pressed the button?”
- Troubleshoot circuits and celebrate small discoveries.

Expected From Students:

- Interact with the digital circuit and press the virtual button.
- Observe if the LED responds.
- Record what they see (e.g., “The LED lights up when I press the button.”).
- Begin to form simple ideas about how input/output works.



DIY Lab

Objective:

Students will simulate an interactive Arduino circuit in Tinkercad to observe how a pushbutton input controls an LED output, building understanding of simple conditional logic and physical computing.

Activity link: <https://www.tinkercad.com/things/hfllsXrRM6S/editel?returnTo=%2Fdashboard>



Activity Instructions for Teachers:

1. Introduce the Concept:
 - Say: “Today we’ll look at a circuit that includes a button and an LED. But there’s a twist—it won’t work until we give it the right instructions!”
 - Ask: “What do you think controls when the LED turns on or off?”
2. Open Tinkercad Circuit:
 - Have students open the Tinkercad simulation that shows the Arduino, button, and LED.
 - Ensure the wiring is visible but let students notice the circuit doesn't respond yet.
3. Prompt a Discovery Moment:
 - Ask students to click the button and observe the LED.
 - Then ask:
 - “Why isn’t anything happening?”
 - “Do you think something’s missing?”

4. Reveal the Insight:

- Explain: “Just like our story said—the board is like the brain, but it needs code to work! Without telling it what to do, it just sits there.”

Explain



Teacher's Role:

- Play the video and pause at key points for questioning.
- Use visuals to reinforce vocabulary (code, command, upload).
- Clarify any confusing terms or logic from the video.
- Connect what students saw in Tinkercad to real coding tasks.

Expected From Students:

- Watch attentively and answer reflection questions.
- Learn what programming is and how it helps electronics “follow orders.”
- Relate programming to what they experienced with the LED/button.
- Start understanding that code tells devices what to do.



Screen Lessons

Objective:

Students will understand the basics of programming, how to write simple instructions (code) for controlling hardware (like an LED), and the importance of coding in building interactive projects.

Activity Link: https://www.youtube.com/watch?v=_j4Lj-BT00g



Possible Strategies for implementation:

1. Pre-Watching: Set a Purpose.

- **Strategy:** What is Programming?
- Before showing the video, ask students: “What do you think programming means? Can you think of any examples where we use programming to make something work, like a robot or a video game?”
 - **Objective:** This strategy sets the purpose by engaging students' prior knowledge and helping them connect the idea of programming to everyday experiences.
 - **Teacher's Role:** Write down students' ideas on the board to revisit after the video.
 - **Expected from Students:** Reflect on what they already know and make predictions about how programming might work in their projects.

2. During Watching: Active Engagement.

- **Strategy:** Key Concept Tracker

While watching the video, provide students with a list of key concepts to track: “What does the code do?”, “What happens when we press the button?”, and “What part of the circuit is being controlled?”

- Objective: This strategy encourages students to focus on the main ideas in the video and relate them directly to the circuit they’re working with.
- Teacher's Role: Pause the video occasionally to ask questions like, "What do you think will happen when the code runs?" or “How does this relate to the circuit we are building?”
- Expected from Students: Actively listen for answers to these questions and take note of any new concepts introduced in the video.

3. Post-Watching: Discussion and Reflection.

- Strategy 1: Code Reflection

After watching the video, discuss with students: “How does programming help us control a device like the LED? What might happen if the code wasn’t written correctly?”

- Objective: This helps students connect the video content to the real-world application in their projects.
- Teacher's Role: Facilitate a class discussion about how programming affects the behavior of the circuit and help students articulate their thoughts.
- Expected from Students: Share ideas on how programming interacts with hardware and reflect on why code is necessary to make things work.

- Strategy 2: Design Your Program

Ask students: “Based on what you learned from the video, how would you write a program to control the LED and make it turn on and off with a button?”

- Objective: This encourages creative thinking as students begin to visualize and design their own program.
- Teacher's Role: Encourage students to share ideas and provide feedback on their designs.
- Expected from Students: Sketch or describe their program ideas, thinking about how they would control an LED or button in their own projects.

- Strategy 3: Apply What You’ve Learned

Once students are comfortable with the basics, give them a challenge: “Create a program for a new button-controlled project. What will it do when pressed?”

- Objective: This extends students' learning by providing an opportunity to apply the concepts in a hands-on way.
- Teacher's Role: Support students as they write their programs, providing assistance if needed.
- Expected from Students: Write or sketch a simple program idea and share it with the class or small groups.

Elaborate



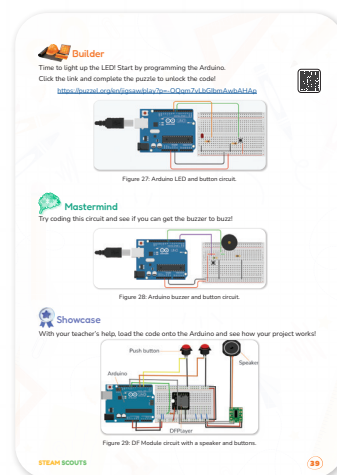
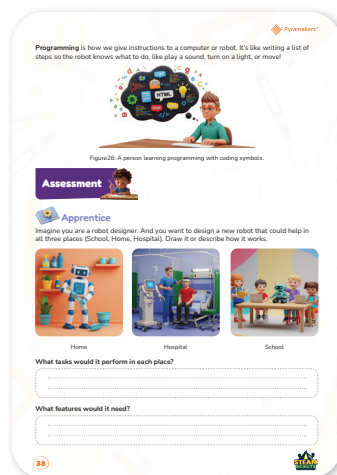
Teacher's Role

- Prompt students to design a robot for different settings (Home, School, Hospital).
- Encourage imagination and practical thinking about robot features.
- Provide the puzzle link and supervise as they solve it and apply code.
- For Mastermind, support students in adjusting buzzer code and testing circuits.

Expected from Students

- Design or describe their own assistive robot with clear functions.
- Complete the coding puzzle and unlock LED code.
- Upload code and test whether the LED lights up.
- Try modifying code to get a buzzer to sound and debug errors.

Parts of the books included.



Apprentice

Objective:

Students will demonstrate understanding of how robots are used in different real-world settings by matching each robot to the correct location based on its design and task.

Correct Answers:

Robot Tasks in Home, Hospital, and School:

- Home: The robot could help with chores like cleaning, cooking, or organizing. It could assist elderly family members or provide companionship to those living alone.
- Hospital: The robot might deliver medications, assist in patient care, or help transport items between departments. It could also monitor vital signs or help doctors and nurses with tasks.

- **School:** The robot could help with teaching activities like reading to students, helping them complete assignments, or assisting students with disabilities. It could also manage classroom tasks or assist in providing support for teachers.

Features the Robot Needs:

- **Mobility:** The robot should be able to move around different environments (home, hospital, and school).
- **Sensors:** Sensors to detect obstacles, monitor health stats (for hospitals), or detect items and people.
- **Communication:** The ability to talk, listen, and understand voice commands.
- **Learning Capabilities:** AI-powered features that enable it to learn from interactions and adapt to different tasks.
- **Safety Features:** Safe for use around people and equipped with emergency features.
- **Tools for Specific Tasks:** For example, cleaning tools at home, medical tools in a hospital, and educational tools in a classroom.

Possible Strategies for Implementation:

1. Activate Prior Knowledge:

- Begin by discussing the concept of robots and where we see them in everyday life (e.g., at home, in hospitals, and in schools). This will help students connect to the real-world relevance of the task.
- Ask students to recall any robots they know and the tasks they perform in these settings. Have students share their experiences and ideas.

2. Use Visual Stimuli:

- Display the images (of home, hospital, and school) and ask students to look closely at the settings. Encourage them to think about how robots might assist in each of these environments.
- Use a whiteboard or digital tool to draw a Venn diagram for students to compare how robots in different settings might have overlapping and unique features.

3. Facilitate Group Brainstorming:

- Pair or group students to brainstorm possible tasks a robot could perform in each setting. Provide guiding questions:
 - What are common needs in homes, hospitals, and schools that a robot could address?
 - How could a robot be useful in emergency situations or to assist people with disabilities?

4. Encourage Creative Design:

- After brainstorming, have students either sketch or describe their robot design. Encourage creativity by asking them to think about what features their robot needs (e.g., sensors, wheels, hands) to perform its tasks effectively in each environment.
- Give students tools like drawing supplies or digital tools to represent their ideas.

5. Prompt Reflection and Sharing:

- Once students have completed their designs, invite them to present their ideas to the class. Ask them to explain the tasks their robot would perform and why those tasks are important.
- Use a discussion prompt: “How would you improve your design if you had more resources or knowledge of robot features?”

6. Integration with Technology:

- If possible, integrate technology by allowing students to create a simple digital mock-up of their robot using basic design software, apps, or drawing tools.
- Introduce programming concepts and connect the robot design with the idea of coding by discussing how commands might be given to robots to make them perform tasks.



Builder

Objective:

Students will solve a digital puzzle to reveal a block-based program. Then, using Tinkercad, they will simulate an Arduino circuit where pressing a button triggers an LED to light up or a sound to play — applying their understanding of coding and hardware interaction.

Materials Needed:

- Devices with internet access (laptops/tablets)
- Student Tinkercad accounts (set up in advance)
- Puzzle link: <https://puzzel.org/en/jigsaw/play?p=-OQqm7vLbGIbmAwbAHAp>
- Tinkercad circuit workspace with Arduino, LED, button, and resistor
- Provided block code (from the puzzle image)



Activity Instructions:

1. Solve the Puzzle.

- Students go to the link and complete the jigsaw puzzle. The final image reveals block code for controlling an LED with a button.

2. Open Tinkercad.

- Students log into Tinkercad and open a provided circuit template.

3. Enter the Code.

- In the Code tab, students switch to Blocks + Text view and recreate the code shown in the puzzle image.

4. Run the Simulation.

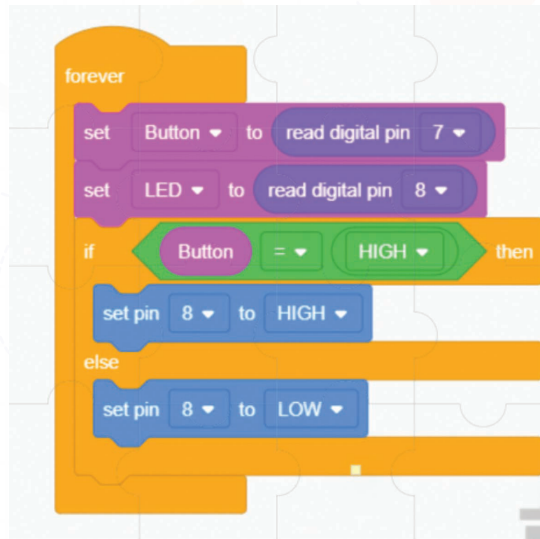
- Students press the button in the simulator and observe if the LED responds correctly.

5. Troubleshoot if Needed.

- Students check if their wiring matches the template and if the blocks match the correct logic.

6. Document and Reflect

- Students draw a diagram of the working circuit or take a screenshot, and explain how the program controls the LED.



Possible Strategies for Implementation:

- Collaborative Puzzle Solving
 - Let students work in pairs to solve the puzzle and build the circuit together.
- Template Support
 - Share a pre-made Tinkercad circuit to focus on programming, or allow advanced students to build from scratch.
- Step-by-Step Code Walkthrough
 - Use a projector to walk students through each block, comparing it to the puzzle solution.
- Error Catchers
 - Provide a checklist of common issues (e.g., incorrect pin numbers, missing brackets, incomplete logic).
- Reflective Prompt
 - Ask: “What part of your code tells the Arduino when to light the LED?”



Mastermind

Objective:

Students will independently apply their understanding of coding and circuits by attempting to program a buzzer to respond to a button press using Tinkercad. This activity reinforces problem-solving, experimentation, and confidence in using the Arduino as a programmable brain.

Note: This activity can be completed during the session if time allows or assigned as an optional take-home challenge.

Activity Instructions:

The same previous code can be applied for this circuit but will just change the variable name to buzzer instead of LED.

Possible Strategies for Implementation:

1. Offer a template (optional)

- Provide a partially built Tinkercad circuit so students can focus on writing and adjusting the code rather than wiring from scratch.

2. Use Think-Aloud Modeling

- For students who need support, walk through a similar example (e.g., turning on an LED with a button) before they try the buzzer independently.

3. Encourage sharing

- Let students who complete the challenge demonstrate their working circuits to the class or small groups.

4. Promote take-home extension

- Allow students to revisit the challenge at home and share screenshots or circuit drawings during the next session.

5. Use reflection prompts

- Ask students to respond to a prompt like:
 - “What was your biggest challenge in getting the buzzer to work, and how did you solve it?”

Evaluate



Teacher's Role

- Help students upload their full code to Arduino.
- Supervise connections and safety.
- Provide feedback and ask questions like “What makes it work?”
- Celebrate success and help students reflect on challenges they overcame.

Expected From Students

- Follow instructions and upload their code.
- Connect components accurately (speaker, button, sound module).
- Test and share their working project.
- Reflect on what they built, how it works, and what they would improve.



Showcase

Objective:

Observe the final step of the project and understand how the Arduino functions with the code uploaded, including how the button controls the LED and the buzzer.

Activity Instructions for Teachers:

1. Prepare the Arduino Setup:

- Ensure the Arduino board is correctly wired with the buttons, LED, and buzzer as shown in the diagram.
- Make sure the students have already completed the previous steps (e.g., designing the circuit, coding, testing).

2. Upload the Code to the Arduino:

- With your students' attention, explain the function of the code they will upload onto the Arduino.
- Demonstrate how to upload the code to the Arduino board using the Arduino IDE (Integrated Development Environment).

CODE Details for teachers only

Arduino file: <https://drive.google.com/drive/folders/1gOq3HH4uOZ8uTQnjtUQCy8L6nwiOOZ>



Library Includes

cpp

CopyEdit

```
#include <SoftwareSerial.h>
```

```
#include <DFRobotDFPlayerMini.h>
```

- SoftwareSerial.h: Enables serial communication on other digital pins (useful when hardware serial is occupied).
- DFRobotDFPlayerMini.h: Provides functions to control the MP3 module (play, stop, set volume, etc.).

Pin Definitions

cpp

CopyEdit

```
#define MP3_RX 10 // Connects to DFPlayer's TX pin
```

```
#define MP3_TX 11 // Connects to DFPlayer's RX pin
```

```
#define BTN_1 4 // Button 1 (Track 1)
```

```
#define BTN_2 6 // Button 2 (Track 2)
```

- MP3_RX & MP3_TX: Used by SoftwareSerial to communicate with the DFPlayer Mini.
- BTN_1 & BTN_2: Digital input pins connected to buttons that play different tracks.

Object Initialization

cpp

CopyEdit

```
SoftwareSerial mp3Serial(MP3_RX, MP3_TX); // Software Serial for DFPlayer
DFRobotDFPlayerMini myDFPlayer;
```

- mp3Serial: A virtual serial connection for DFPlayer communication.
- myDFPlayer: The DFPlayer Mini controller object.

Debouncing Variables

cpp

CopyEdit

```
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50; // 50ms delay
int currentTrack = 0; // 0 = stopped, 1 = track 1, 2 = track 2
```

- Debouncing: Prevents multiple activations from a single button press.
- currentTrack: Tracks the currently playing audio to avoid replaying the same track unnecessarily.

Setup Function

cpp

CopyEdit

```
void setup() {
  pinMode(BTN_1, INPUT_PULLUP);
  pinMode(BTN_2, INPUT_PULLUP);

  mp3Serial.begin(9600);
  myDFPlayer.begin(mp3Serial);

  myDFPlayer.volume(30);           // Volume: 0 (min) to 30 (max)
  myDFPlayer.outputDevice(DFPLAYER_DEVICE_SD); // Use SD card
  myDFPlayer.EQ(DFPLAYER_EQ_NORMAL); // Equalizer: Normal
}
```

- INPUT_PULLUP: Eliminates the need for external resistors.
- DFPlayer Settings:
 - Volume is set to max (30).
 - Audio source is the SD card.
 - Equalizer is set to "Normal" (can also be POP, ROCK, etc.).

Main Loop (Button Handling & Playback)

cpp

CopyEdit

```
void loop() {  
  if ((millis() - lastDebounceTime) > debounceDelay) {  
    bool btn1Pressed = !digitalRead(BTN_1); // LOW = pressed  
    bool btn2Pressed = !digitalRead(BTN_2); // LOW = pressed  
  
    if (btn1Pressed && currentTrack != 1) {  
      myDFPlayer.play(1); // Play 0001.mp3  
      currentTrack = 1;  
      lastDebounceTime = millis();  
    }  
    else if (btn2Pressed && currentTrack != 2) {  
      myDFPlayer.play(2); // Play 0002.mp3  
      currentTrack = 2;  
      lastDebounceTime = millis();  
    }  
    else if (!btn1Pressed && !btn2Pressed && currentTrack != 0) {  
      myDFPlayer.stop(); // Stop playback  
      currentTrack = 0;  
    }  
  
    delay(10); // Reduce CPU usage  
  }  
}
```

How It Works

- Button Detection:
 - Buttons are active LOW due to INPUT_PULLUP.
 - If Button 1 is pressed, Track 1 plays (0001.mp3).
 - If Button 2 is pressed, Track 2 plays (0002.mp3).
- Debouncing:
 - Ensures 50ms must pass before another button press is accepted.
- Auto Stop:
 - If neither button is pressed and audio is playing, it stops playback.

Upload the Code to the Arduino:

- With your students' attention, explain the function of the code they will upload onto the Arduino.
- Demonstrate how to upload the code to the Arduino board using the Arduino IDE (Integrated Development Environment).
- This video can help you to understand how to do this step:
<https://www.youtube.com/watch?v=xQfC72VeV7Y>



Demonstrate the Functioning Circuit:

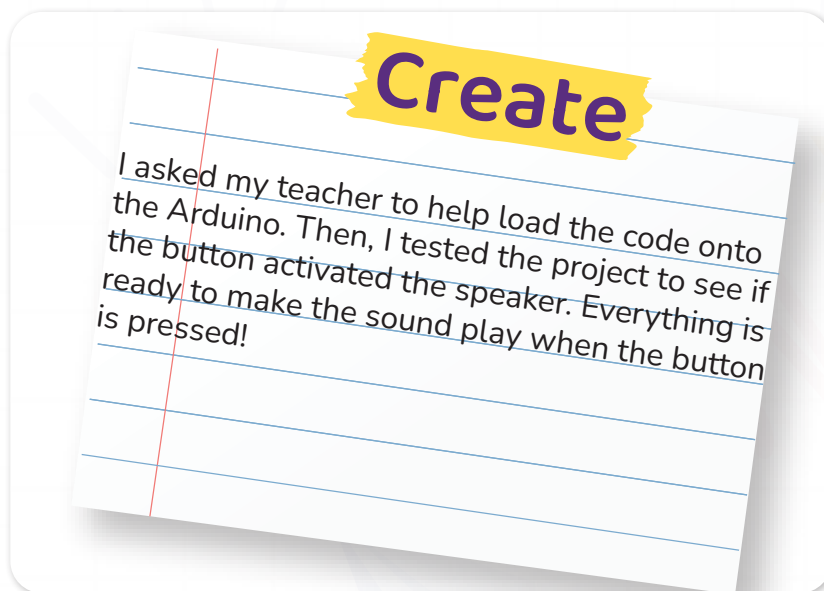
- Once the code is uploaded, press the buttons to show how the circuit works.

Note: Setup Dfmodule

1. Download or record sound in MP3
2. Load the sounds files on the SD card

Note that the files 0001.mp3 , 0002.mp3

It's time to move to the Create part and add this.



Now I Can

Objective:

Students will reflect on their learning by identifying key concepts they've mastered related to robots, Arduino programming, and how technology can be used to assist people in various settings (home, school, hospital). They will also apply their understanding of block coding to create a talking device that responds to button presses.

Activity Instructions

1. Introduce the Checklist:

- Say: "Let's think back on everything we learned about sound today—how it moves, how it helps people, and how we explored it in class."

2. Guide the Students Through the Items:

- Read each “Now I Can” statement aloud one at a time.
- Ask: “Can you give a thumbs up if you feel proud of this learning?”
- Encourage brief discussion or examples after each one.

3. Have Students Complete Their Checklists:

- Let students place a checkmark next to what they feel confident about.
- Remind them: “It’s okay if you don’t check everything—we’re all still learning!”

4. Optional Extension:

- Ask students to circle the one they’re most proud of.
- Let them draw or write about it in the bottom of the page.

Relation to Standards:

1. NGSS – Next Generation Science Standards

3-5-ETS1-1:

Define a simple design problem reflecting a need or a want that includes specified criteria for success and constraints on materials.

(Students design and test a talking device to meet a communication need.)

3-PS2-4:

Define a simple design problem that can be solved by applying scientific ideas about electric forces.

(Understanding how pressing a button sends an electrical signal that triggers a programmed response.)

2. CCSS – Common Core State Standards (Mathematics)

3.OA.A.3:

Use multiplication and division within 100 to solve word problems in situations involving measurement quantities.

(Applied when students consider circuits, pins, voltage needs, or program loops.) 3.MD.A.2: Measure and estimate liquid volumes and masses of objects using standard units. (Supports logical thinking and estimation used in programming and debugging.)

3. ISTE – International Society for Technology in Education

1.4 Innovative Designer:

Students use a variety of technologies within a design process to identify and solve problems by creating new, useful, or imaginative solutions.

(Programming a device to play different sounds when buttons are pressed.)

1.5 Computational Thinker:

Students develop and employ strategies for understanding and solving problems in ways that leverage the power of technological methods.